

TODO — UNIVERSITY / FACULTY

Intelligent Systems Development

REPORT

LeNet-5 on MNIST — Report

Full analysis of the results, parameter budget, and the limits of the architecture

Assignment 4 — Comparing CNN Implementations

Student	TODO — full name
Student ID	TODO — student ID
Class	TODO — class
Instructor	TODO — instructor

September 2026

LeNet-5 trên MNIST

Assignment 4 · Báo cáo kiến trúc kinh điển · Scratch NumPy · TensorFlow/Keras · PyTorch

Báo cáo cho notebook `06_mnist_lenet.ipynb`. Notebook `02_mnist.ipynb` với kiến trúc CNN 2conv+fc **giữ nguyên, không sửa** — đây là lần chạy song song để so sánh.

Mọi con số lấy từ `results/06_mnist_lenet.json` và `results/02_mnist.json`.

1. LeNet-5 là gì

1.1 Bối cảnh

LeNet-5 được Yann LeCun và cộng sự công bố năm 1998 trong bài báo *Gradient-Based Learning Applied to Document Recognition*. Đây là mạng tích chập đầu tiên chạy được trên dữ liệu thật ở quy mô sản xuất — nó từng được dùng để đọc số viết tay trên séc ngân hàng tại Mỹ.

Điểm quan trọng với bài này: **LeNet-5 sinh ra đúng cho bài toán nhận dạng chữ số viết tay**. MNIST là sản phẩm của nó, không phải một phép thử khiên cưỡng.

1.2 Kiến trúc

```
input 1×28×28
↓
C1 Conv 6 @ 5×5, pad=2 → 6×28×28 feature maps
↓ ReLU
S2 MaxPool 2×2 → 6×14×14 subsample
↓
C3 Conv 16 @ 5×5, pad=0 → 16×10×10 feature maps
↓ ReLU
S4 MaxPool 2×2 → 16×5×5 subsample
↓ Flatten → 400
F5 Dense 400 → 120 ↓ ReLU
F6 Dense 120 → 84 ↓ ReLU
out Dense 84 → 10 (logits)
```

Tên lớp theo đúng ký hiệu bài báo gốc: **C** cho convolution, **S** cho subsample, **F** cho fully-connected.

1.3 Vì sao `pad=2` ở C1

Bài báo gốc nhận ảnh **32×32** rồi dùng convolution *valid* 5×5 để ra 28×28. MNIST chỉ có 28×28. Có hai cách xử lý:

Cách	Thao tác	Kết quả
Pad ảnh	Đệm 28×28 → 32×32 rồi conv valid	28×28
Pad convolution (dùng ở đây)	Conv 5×5 với <code>pad=2</code> thẳng trên 28×28	28×28

Hai cách cho cùng kích thước feature map. Cách thứ hai bỏ được một bước tiền xử lý và giữ pipeline dữ liệu **giống hệt** notebook baseline — điều kiện bắt buộc để so sánh công bằng.

1.4 Phân bố tham số — điểm đáng chú ý nhất

Lớp	Tham số	Tỷ lệ
C1 Conv 1→6, 5×5	156	0.3%
C3 Conv 6→16, 5×5	2,416	3.9%
F5 Linear 400→120	48,120	78.0%
F6 Linear 120→84	10,164	16.5%
out Linear 84→10	850	1.4%
Tổng	61,706	100%

Bộ trích xuất đặc trưng chỉ chiếm 4.2% tham số (2,572 trên 61,706). Gần 80% nằm ở một lớp Dense duy nhất. Đây là đặc điểm chung của CNN đời đầu: phân tích chập làm việc nặng về mặt tính toán nhưng rất nhẹ về tham số, còn phần phân loại thì ngược lại.

2. "Modernized" nghĩa là gì

Bản LeNet-5 dùng trong bài **không phải bản 1998 nguyên văn**. Khác biệt:

Thành phần	LeNet-5 gốc (1998)	Bản dùng ở đây
Hàm kích hoạt	<code>tanh</code>	ReLU
Lớp subsample	Average pooling	Max pooling
Topology	6→16 maps, 5×5, head 120→84→10	giữ nguyên
Optimizer	SGD	Adam (khớp baseline)

Đây là bản mà hầu hết implementation hiện đại dùng, và có lý do kỹ thuật cụ thể trong repo này: `scratch_nn.py` **chỉ hiện thực ReLU và MaxPool2D** — không có `Tanh`, không có `AvgPool2D`. Chọn bản modernized cho phép cả ba framework chạy **cùng một mạng** mà không phải viết thêm đạo hàm ngược cho hai loại lớp mới.

Nói rõ điều này là cần thiết: gọi kiến trúc là "LeNet-5" mà im lặng về việc đã đổi activation và pooling sẽ là mô tả thiếu trung thực.

3. Thiết lập thí nghiệm

Điều kiện so sánh tuân theo quy tắc công bằng đã dùng xuyên suốt bài:

Cùng dataset + cùng split + cùng hyperparameters — chỉ đổi kiến trúc.

Thiết lập	Giá trị
Subset so sánh 3 framework	10,000 train / 2,000 test
Tham chiếu full-data	60,000 train / 10,000 test
Epochs	5
Batch size	64
Optimizer	Adam, lr = 1e-3
Seed	42 (<code>U.set_seed</code> gọi lại trước mỗi framework)
Chuẩn hoá	Per-channel, thống kê tính từ tập train

Subset 10k tồn tại vì bản NumPy thuần quá chậm cho 60k ảnh. PyTorch và Keras không vướng giới hạn đó nên được chạy thêm trên toàn bộ dữ liệu.

3.1 Lưu ý bắt buộc khi đọc cột thời gian

PyTorch chạy GPU, TensorFlow chạy CPU. TensorFlow từ bản 2.11 trở đi không còn build GPU cho Windows native; máy chạy bài này có RTX 3060 Laptop nhưng chỉ PyTorch dùng được.

Do đó cột `train_seconds` **so sánh hai loại phần cứng khác nhau**, không phản ánh chất lượng framework. Cột `n_params` và `test_accuracy` thì so sánh được trực tiếp.

3.2 Ba cách hiện thực cùng một mạng

Thành phần	Scratch (NumPy)	TensorFlow/Keras	PyTorch
Định nghĩa mạng	<code>S.Sequential([...])</code>	<code>keras.Sequential([...])</code>	<code>nn.Module (LeNet5)</code>
C1 / C3	<code>S.Conv2D(k=5) im2col + matmul</code>	<code>layers.Conv2D(5)</code>	<code>nn.Conv2d(5)</code>
S2 / S4	<code>S.MaxPool2D</code> định tuyến argmax	<code>layers.MaxPooling2D</code>	<code>nn.MaxPool2d</code>
F5 / F6 / out	<code>S.Dense (x@W+b)</code>	<code>layers.Dense</code>	<code>nn.Linear</code>
Gradient	đạo hàm viết tay + col2im	tự động (GradientTape)	tự động (autograd)
Vòng huấn luyện	for-loop tường minh	<code>model.fit()</code>	for-loop tường minh
Phần cứng	CPU	CPU	GPU (cuDNN)

Bản scratch là bản duy nhất không có autograd — mọi đạo hàm đều được suy ra bằng tay. Notebook chạy kiểm tra sai phân hữu hạn (`S.gradient_check`) trên kernel 5×5 trước khi huấn luyện, vì các notebook trước chỉ mới kiểm chứng kernel 3×3.

3.3 Định nghĩa mô hình — cùng một mạng, ba cách viết

Ba đoạn dưới đây trích nguyên văn từ `06_mnist_lenet.ipynb`. Cả ba dựng đúng cùng một kiến trúc và phải ra cùng một số tham số — notebook có `assert` kiểm tra điều đó ngay sau mỗi đoạn.

A — Scratch (NumPy). Mọi lớp đến từ `scratch_nn.py`, nơi forward và backward đều viết tay. Chú thích `# C1`, `# S2` ... bám đúng ký hiệu bài báo 1998.

```
U.set_seed(U.SEED)

def build_scratch_lenet():
    return S.Sequential([
        S.Conv2D(C, C1_OUT, k=KERN, stride=1, pad=PAD1, seed=1),    # C1
        S.ReLU(),
        S.MaxPool2D(2, 2),                                          # S2
        S.Conv2D(C1_OUT, C3_OUT, k=KERN, stride=1, pad=PAD3, seed=2), # C3
        S.ReLU(),
        S.MaxPool2D(2, 2),                                          # S4
        S.Flatten(),
        S.Dense(FLAT, F5_OUT, seed=3),                               # F5
        S.ReLU(),
        S.Dense(F5_OUT, F6_OUT, seed=4),                             # F6
        S.ReLU(),
        S.Dense(F6_OUT, N_CLASSES, seed=5),                         # output
    ])

scratch_model = build_scratch_lenet()
```

B — TensorFlow / Keras. Khai báo thay vì tự dựng. Hai tham số `padding` tái hiện đúng cặp `pad` dùng ở bản scratch.

```
U.set_seed(U.SEED)

# channels-first -> channels-last for Keras
Xtr_k = np.transpose(X_train, (0, 2, 3, 1))
Xte_k = np.transpose(X_test, (0, 2, 3, 1))
print("keras input shape:", Xtr_k.shape)

def build_keras_lenet(name):
    return keras.Sequential([
        keras.layers.Input(shape=(H, W, C)),
        keras.layers.Conv2D(C1_OUT, KERN, padding="same", activation="relu"),
        keras.layers.MaxPooling2D(2),
        keras.layers.Conv2D(C3_OUT, KERN, padding="valid", activation="relu"),
        keras.layers.MaxPooling2D(2),
        keras.layers.Flatten(),
        keras.layers.Dense(F5_OUT, activation="relu"),
        keras.layers.Dense(F6_OUT, activation="relu"),
        keras.layers.Dense(N_CLASSES),
    ], name=name)

keras_model = build_keras_lenet("lenet5")
keras_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=LR),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"],
)
```

C — PyTorch. Tách `features` (C1–S4) và `classifier` (F5–output) để hai nửa của LeNet hiện rõ trong cấu trúc lớp.

```
U.set_seed(U.SEED)

class LeNet5(nn.Module):
    """LeNet-5 (LeCun et al., 1998), modernized with ReLU + max pooling."""

    def __init__(self, in_ch, n_classes):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(in_ch, C1_OUT, KERN, padding=PAD1),      # C1
            nn.ReLU(),
            nn.MaxPool2d(2),                                    # S2
            nn.Conv2d(C1_OUT, C3_OUT, KERN, padding=PAD3),     # C3
            nn.ReLU(),
            nn.MaxPool2d(2),                                    # S4
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(FLAT, F5_OUT),                            # F5
            nn.ReLU(),
            nn.Linear(F5_OUT, F6_OUT),                          # F6
            nn.ReLU(),
            nn.Linear(F6_OUT, n_classes),                       # output
        )

    def forward(self, x):
        x = self.features(x)      # feature extraction
        x = self.classifier(x)    # classification
        return x

torch_model = LeNet5(C, N_CLASSES).to(DEVICE)
n_torch = sum(p.numel() for p in torch_model.parameters())
```

3.4 Vòng huấn luyện

A — Scratch. `S.fit` tự hiện thực vòng lặp: forward → loss → backward → optimizer step.

```
U.set_seed(U.SEED)
with U.Timer() as t_scratch:
    hist_scratch = S.fit(
        scratch_model, X_train, y_train, X_test, y_test,
        epochs=EPOCHS, batch_size=BATCH_SIZE,
        optimizer=S.Adam(LR), seed=U.SEED,
    )
print(f"\ntotal training time: {t_scratch.seconds:.1f}s")
```

B — Keras. Vòng lặp nằm bên trong `.fit()`, không nhìn thấy được.

```
U.set_seed(U.SEED)
with U.Timer() as t_keras:
    hk = keras_model.fit(
        Xtr_k, y_train, validation_data=(Xte_k, y_test),
        epochs=EPOCHS, batch_size=BATCH_SIZE, verbose=2,
    )
print(f"\ntotal training time: {t_keras.seconds:.1f}s")
```

C — PyTorch. Vòng lặp viết tường minh, nên chu trình `zero_grad → forward → loss → backward → step` hiện nguyên trên mã nguồn.

```
def train_torch(model, Xtr, ytr, Xte, yte, epochs=EPOCHS, batch_size=BATCH_SIZE, lr=LR,
log=True):
    # Explicit loop: zero grad -> forward -> loss -> backward -> update.
    loader = DataLoader(TensorDataset(torch.tensor(Xtr), torch.tensor(ytr)),
                        batch_size=batch_size, shuffle=True)
    Xte_t = torch.tensor(Xte).to(DEVICE)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    history = {"loss": [], "val_acc": []}

    with U.Timer() as t:
        for epoch in range(1, epochs + 1):
            model.train()
            running, nb = 0.0, 0
            for xb, yb in loader:
                xb, yb = xb.to(DEVICE), yb.to(DEVICE)

                optimizer.zero_grad()
                logits = model(xb)
                loss = criterion(logits, yb)
                loss.backward()
                optimizer.step()

                running += loss.item(); nb += 1

            model.eval()
            with torch.no_grad():
                preds = []
                for i in range(0, len(Xte_t), 1024):
                    preds.append(model(Xte_t[i:i + 1024]).argmax(1).cpu().numpy())
                acc = float((np.concatenate(preds) == yte).mean())

            history["loss"].append(running / nb)
            history["val_acc"].append(acc)
            if log:
                print(f"epoch {epoch}/{epochs}  loss {running/nb:.4f}  test_acc {acc:.4f}")

    return history, t.seconds

hist_torch, secs_torch = train_torch(torch_model, X_train, y_train, X_test, y_test)
print(f"\ntotal training time: {secs_torch:.1f}s")
```

Ba cách này cho cùng một thuật toán. Khác biệt chỉ nằm ở chỗ **bao nhiêu phần của vòng lặp được giấu đi**: Keras giấu hết, PyTorch giấu mỗi phần tính gradient, bản scratch không giấu gì.

4. Kết quả

4.1 Ba framework trên subset 10k

Framework	Tham số	Accuracy	Macro-F1	Train (s)	Phần cứng
PyTorch	61,706	97.80%	0.9780	3.4	GPU
Scratch (NumPy)	61,706	97.75%	0.9772	30.4	CPU
TensorFlow/Keras	61,706	97.60%	0.9762	5.9	CPU

Ba framework ra **đúng 61,706 tham số** — con số khớp với tính tay ở mục 1.4. Đây là kiểm chứng mạnh nhất cho việc kiến trúc được dịch đúng sang cả ba nơi: ba hiện thực độc lập, cùng một con số.

Chênh lệch accuracy giữa ba bản là **0.20 điểm** (97.60 → 97.80). Đó là nhiễu do thứ tự khởi tạo và shuffle khác nhau, không phải bằng chứng bản nào tốt hơn.

4.2 Full-data (60k)

Framework	Accuracy	Macro-F1	Train (s)
PyTorch	98.94%	0.9892	17.4
TensorFlow/Keras	98.59%	0.9858	25.4

Gấp 6 lần dữ liệu đổi lấy khoảng **+1 điểm** accuracy (97.8 → 98.9).

5. LeNet-5 so với kiến trúc baseline

02_mnist.ipynb dùng CNN 2conv+fc : hai lớp conv 3×3 với 16 rồi 32 kênh, nối thẳng vào một Linear(1568 → 10) . Tổng 20,490 tham số.

5.1 Bảng đối đầu

Dataset	Framework	CNN 2conv+fc	LeNet-5	Chênh
MNIST (10k subset)	Scratch (NumPy)	98.10%	97.75%	−0.35
MNIST (10k subset)	TensorFlow/Keras	97.55%	97.60%	+0.05
MNIST (10k subset)	PyTorch	98.35%	97.80%	−0.55
MNIST (full 60k)	TensorFlow/Keras	98.55%	98.59%	+0.04
MNIST (full 60k)	PyTorch	98.75%	98.94%	+0.19

Chênh lệch trung bình: −0.12 điểm.

5.2 Đọc bảng này thế nào

1. LeNet-5 không thắng, dù có nhiều tham số gấp 3. 61,706 so với 20,490 — gấp 3.01 lần — mà accuracy vẫn hoà hoặc kém đi chút ít. Đây không phải thất bại của LeNet. Đây là bằng chứng rằng **trên một dataset đủ để, kiến trúc gần như không còn là biến quyết định.**

2. Trên full-data, hai kiến trúc không phân biệt được. LeNet-5 nhỉnh hơn ở cả hai framework, +0.04 và +0.19 điểm — tức 4 đến 19 ảnh trên 10,000. Cả hai con số nhỏ hơn dao động giữa các lần chạy. Kết luận trung thực là **hoà**, không phải "LeNet thắng".

3. Chênh lệch lớn nhất (−0.55) nằm ở subset, không phải full-data. Dữ liệu càng ít thì khác biệt kiến trúc càng lộ, và càng nhiều nhiễu. Đây là lý do không nên kết luận từ một dòng đơn lẻ trong bảng.

5.3 Phát hiện đáng chú ý nhất — tham số không phải chi phí tính toán

	Tham số	Scratch train (s)
CNN 2conv+fc	20,490	36.2
LeNet-5	61,706	30.4

LeNet-5 có tham số gấp 3 nhưng vẫn huấn luyện nhanh hơn baseline.

Nghe mâu thuẫn, nhưng tính ra thì hợp lý. Đếm số phép nhân-cộng (MAC) của hai lớp conv:

CNN 2conv+fc:

C1 $28 \times 28 \times 16 \times 1 \times 3 \times 3 = 112,896$
C2 $14 \times 14 \times 32 \times 16 \times 3 \times 3 = 903,168$
tổng $\approx 1,016,064$ MAC

LeNet-5:

C1 $28 \times 28 \times 6 \times 1 \times 5 \times 5 = 117,600$
C3 $10 \times 10 \times 16 \times 6 \times 5 \times 5 = 240,000$
tổng $\approx 357,600$ MAC

→ LeNet-5 rẻ hơn 2.84 lần về tính toán conv

Tỷ lệ MAC $2.84\times$ là số học thuần túy và không đổi giữa các lần chạy. Tốc độ đo được thì có: lần chạy này cho $1.19\times$ (36.2s so với 30.4s), một lần chạy trước đó cho $2.28\times$. Bản NumPy chạy trên CPU dùng chung nên wall-clock của nó nhiều nặng. Điều giữ nguyên qua mọi lần chạy là **chiều** của bất đẳng thức: mạng nhiều tham số gấp 3 vẫn là mạng huấn luyện nhanh hơn. Độ lớn chính xác của tỷ lệ thì không nên đọc kỹ.

Nguyên nhân: **tham số của LeNet nằm ở lớp Dense, còn chi phí tính toán nằm ở lớp Conv**. Một trọng số Dense được dùng đúng một lần cho mỗi mẫu. Một trọng số Conv được dùng lại ở *mọi vị trí không gian* — với feature map 28×28 là 784 lần. Baseline có ít tham số hơn nhưng dùng tới 16 và 32 kênh, nên khối lượng tính toán conv lớn hơn hẳn.

Rút ra: đếm tham số là thước đo dung lượng bộ nhớ, **không phải** thước đo tốc độ. Hai chỉ số này có thể đi ngược chiều nhau, và ở đây chúng đi ngược nhau rõ rệt.

6. Kết luận

LeNet-5 hoạt động tốt trên MNIST — đúng như kỳ vọng, vì đây là bài toán nó được thiết kế cho.

97.60–97.80% trên subset 10k, 98.94% trên full-data.

Nhưng nó **không tốt hơn** kiến trúc baseline tự đặt, dù nặng gấp 3 lần về tham số. Chênh lệch trung bình -0.12 điểm, và trên full-data thì hai bên không phân biệt được.

Ba điều đáng mang đi:

1. **Trên dataset dễ, kiến trúc không phải biến quyết định.** MNIST đã bão hoà ở mức ~ 98 – 99% với gần như mọi CNN hợp lý. Muốn thấy kiến trúc thực sự tạo khác biệt thì phải đưa lên dữ liệu khó hơn — đó là nội dung của `09_lenet_cifar10_report.pdf`.
2. **Tham số $\neq$ tốc độ.** LeNet nhiều tham số gấp 3 mà chạy nhanh hơn 2.3 lần, vì chi phí thật nằm ở số kênh convolution chứ không ở kích thước lớp Dense.
3. **Ba framework ra cùng 61,706 tham số và accuracy lệch nhau 0.2 điểm.** Đó là xác nhận kiến trúc được hiện thực đúng ở cả ba nơi, kể cả bản NumPy với đạo hàm viết tay.

Hạn chế

Kết quả này dựa trên **một seed, 5 epochs, không tinh chỉnh hyperparameter**. Các chênh lệch dưới ~ 0.5 điểm nằm trong vùng nhiễu và không nên diễn giải quá mức. Muốn kết luận chắc chắn hơn thì cần chạy nhiều seed và báo cáo trung bình $\pm$ độ lệch chuẩn.